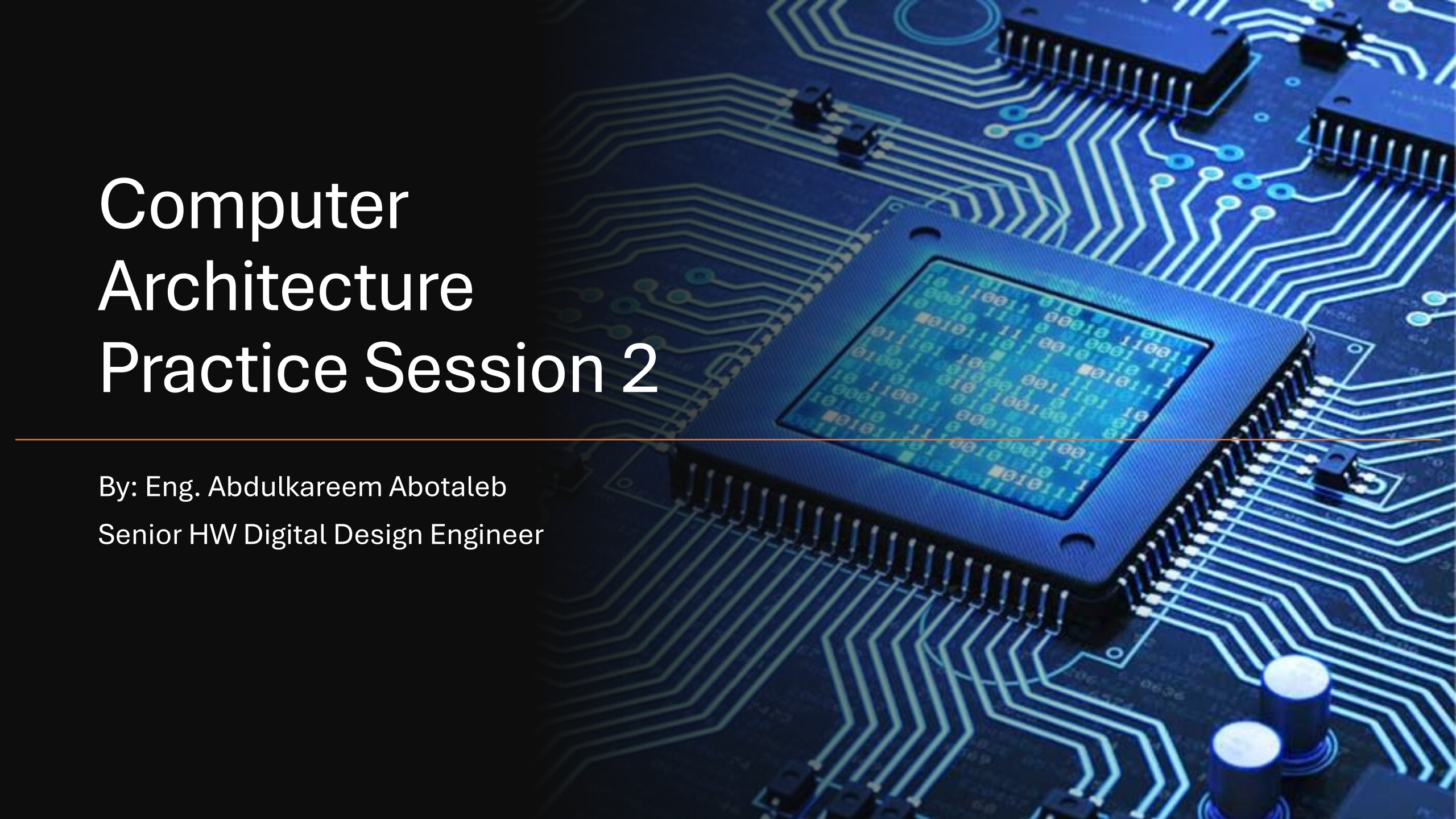
Computer Architecture Practice Session 2



By: Eng. Abdulkareem Abotaleb

Senior HW Digital Design Engineer

Agenda

1. MIPS Basic Architecture

2. MIPS Hierarchy

3. Component Implementation

4. Testbench

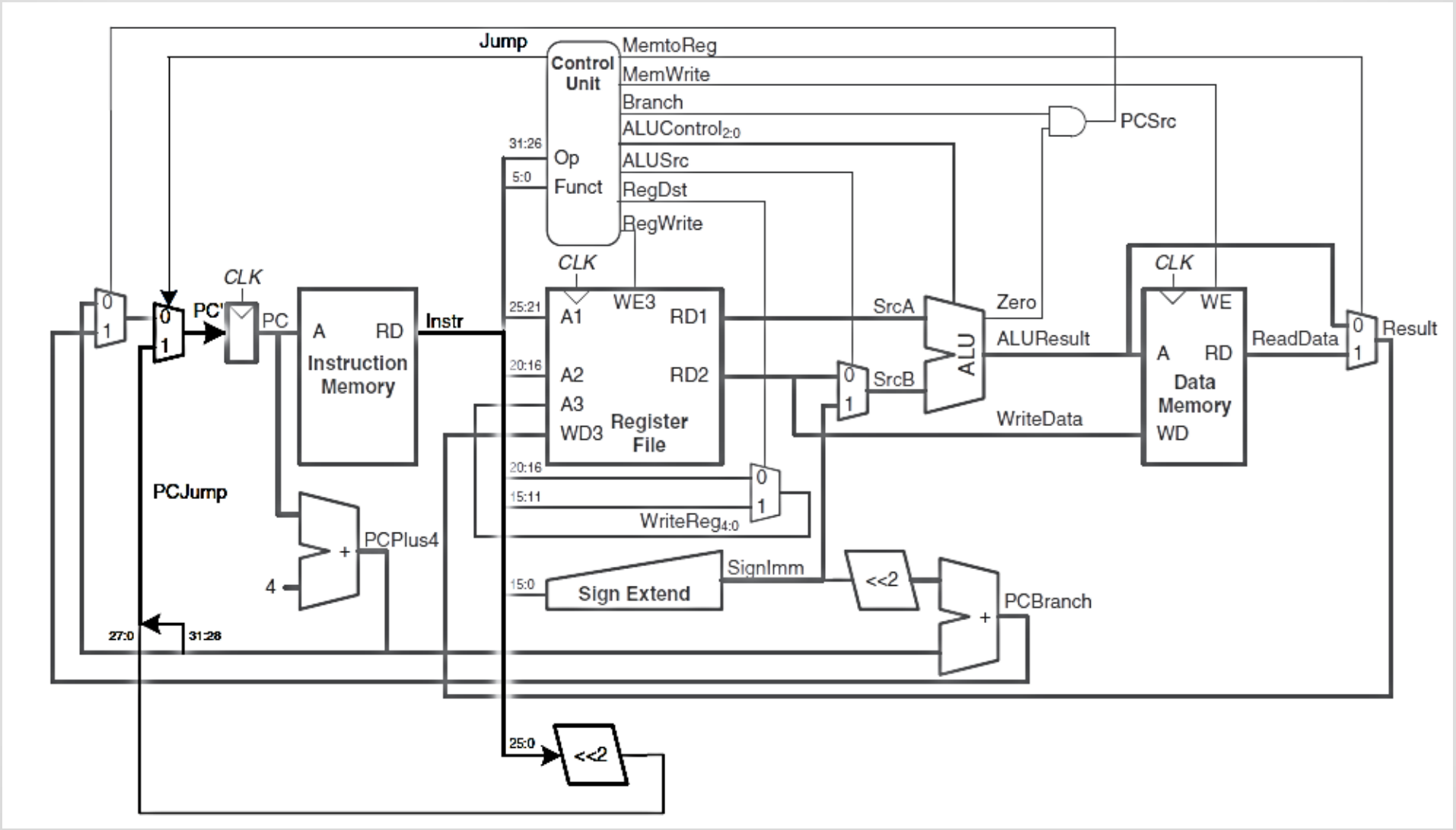
5. Synthesis

Assignment #2

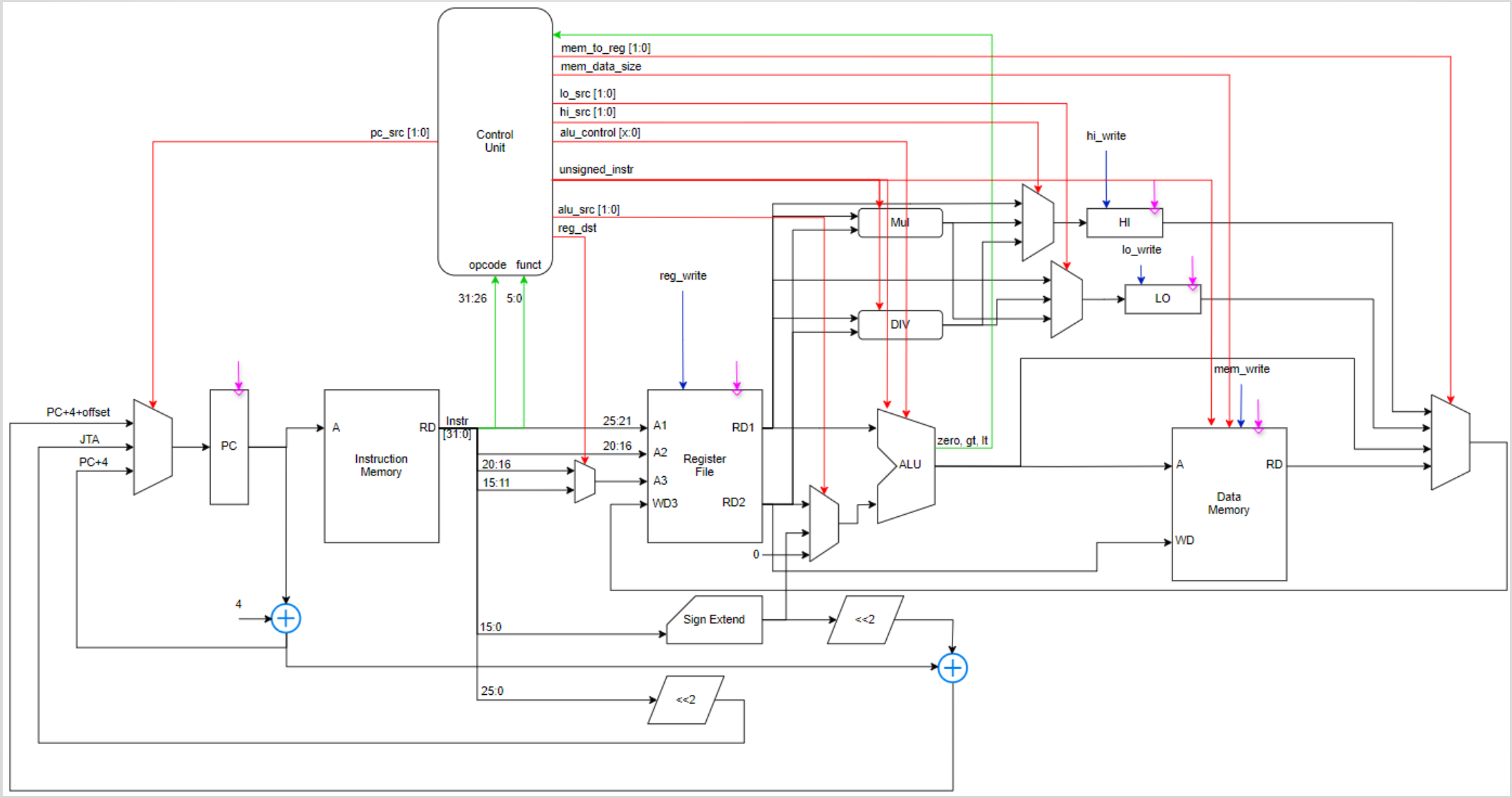
- Read section *7.3 SingleCycle Processor* from the textbook.
- Implement the **full** SingleCycle MIPS Processor which supports all the mentioned instructions, documented in *Appendix B, except floating point instructions*.
- Review and update the single-Cycle MIPS processor.
- Apply the given tests [Test#1,2,5] to verify the functionality.
- [As a plus] Apply a randomized testbench, do the required constraints and the possible assertions
- Deadline: Friday, 1st Nov 2024, at 11:59 PM.
- Deliveries: Design files, testbench, and a presentation for your work.
  Send a mail to: eng.abdulkareem.abotaleb@gmail,com
  Using a title of: CompArch_assignment2_<first_name>_<last_name>

1. MIPS Basic Architecture

1. MIPS Basic Architecture



1. MIPS Single-Cycle Proposed Architecture



2. MIPS Hierarchy

2. MIPS Hierarchy

- mips_tbench
  - mips_top_inst
    - pc_reg_inst
    - instr_mem
    - reg_file_inst
    - alu_inst
    - mult_inst
    - div_inst
    - data_mem
    - ctrl_unit_inst

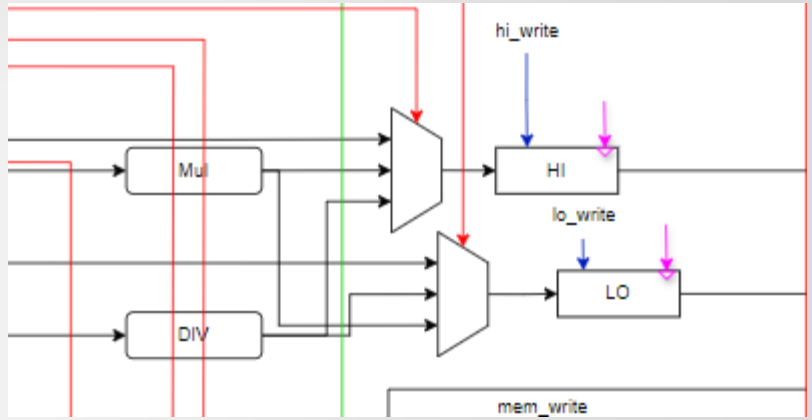
3. Component Implementation

3. Component Implementation

How to implement mthi, mtlo instructions:

- In mips_top

```
assign hi_mux = (hi_src==0)?rs: (hi_src==1)? mul_out: (hi_ src==2)? div_out: rs;
assign lo_mux = (lo_src==0)?rs: (lo_src==1)? mul_out: (lo_ src==2)? div_out: rs;
```



- In control_unit

```
case (opcode)
        mthi_opcode: begin hi_src = 0; hi_write =1; lo_write=0; end
        mtlo_opcode: begin lo_src = 0; hi_write =0; lo_write=1; end
        mult_opcode: begin hi_src = 1; lo_src = 1; hi_write =1; lo_write=1; end
        div_opcode: begin hi_src = 2; lo_src = 2; hi_write =1; lo_write=1; end
        ...
endcase
```

3. Component Implementation

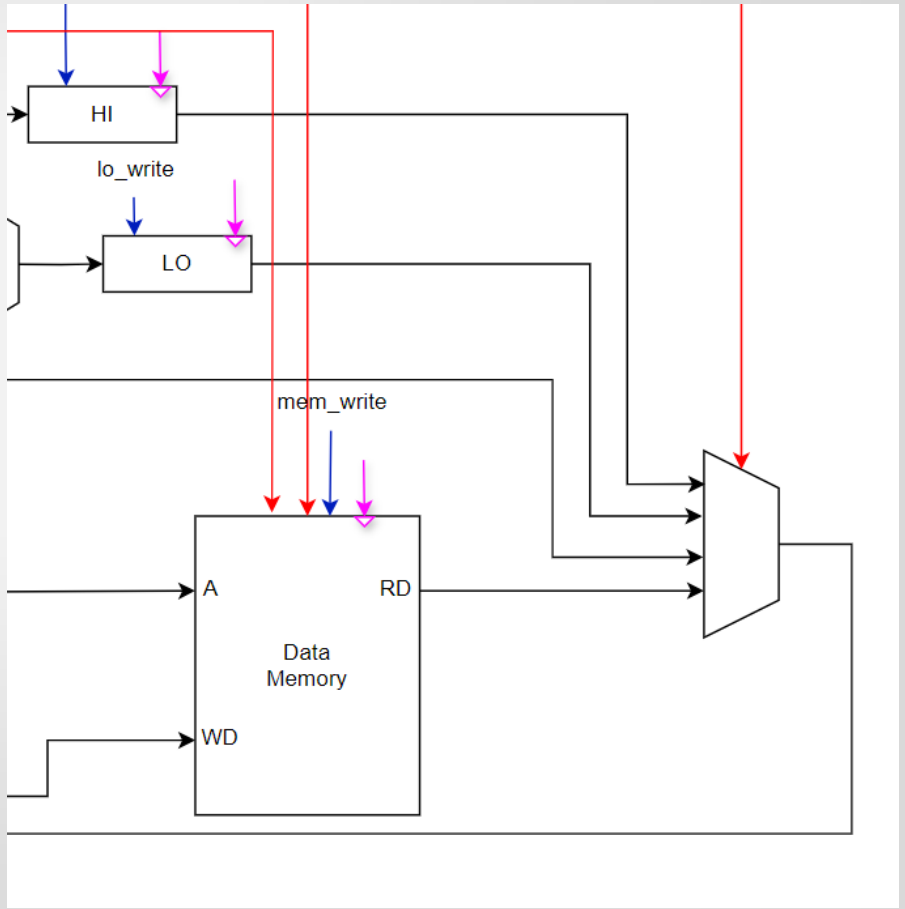
How to implement mfhi, mflo instructions:

- In mips_top

```
assign write_data_rf = (mem_to_reg==0)?hi_reg: (mem_to_reg ==1)? Lo_reg:
(mem_to_reg ==2)? alu_result: rd_mem_data;
```

- In control_unit

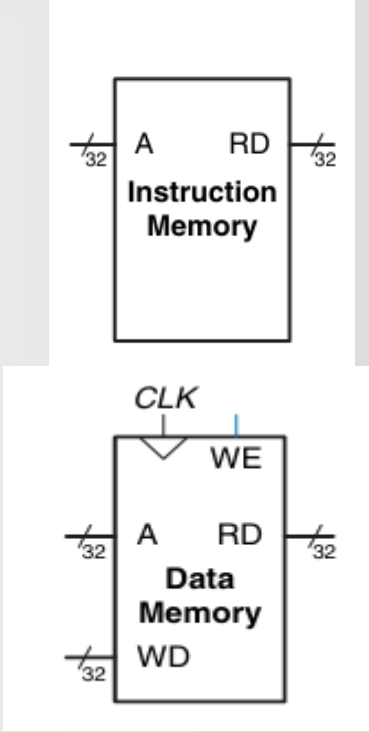
```
case (opcode)
        mfhi_opcode: begin mem_to_reg=0; reg_write=1;end
        mflo_opcode: begin mem_to_reg =1; reg_write=1; end
        lb_opcode: begin mem_to_reg =2; reg_write=1; end
        lh_opcode: begin mem_to_reg=2; reg_write=1; end
        lw_opcode: begin mem_to_reg=2; reg_write=1; end
        remaining_opcodes: begin reg_write=0; end
        …
endcase
```



3. Component Implementation

- Instruction/Data Memory [As implemented in Assignment #1]:

```
module ram_memory #(parameter depth = 1024,  parameter width = 8) (
  //input
  input clk,    input wr_en,
  input [$clog2(depth)-1:0] addr,
  input [(4*width)-1:0] wr_data,
  //output
  output [(4*width)-1:0] rd_data
);
reg [width-1:0] mem [0:depth-1];
initial begin
            for(integer i = 0; i < depth; i = i+1 )
                mem [i] <= 0;
end
always @(posedge(clk)) begin
            if(wr_en)
                {mem [addr+3], mem [addr+2], mem [addr+1], mem [addr]} <= wr_data;
end
assign  rd_data = {mem [addr+3], mem [addr+2], mem [addr+1], mem [addr]};
endmodule
```

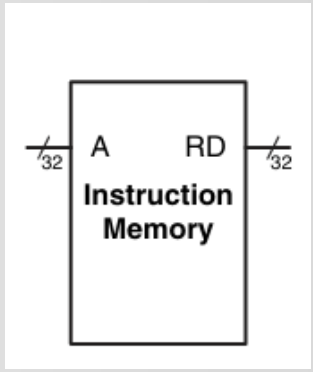


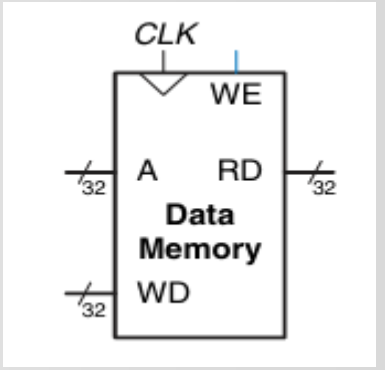
3. Component Implementation

- Instruction/Data Memory:

```
module ram_memory #(parameter depth = 1024,  parameter width = 8) (
  //input
  input clk,    input wr_en,
  input [$clog2(depth)-1:0] addr,
  input [(4*width)-1:0] wr_data,
  input [1:0] mem_data_size,
  //output
  output [(4*width)-1:0] rd_data
);
reg [width-1:0] mem [0:depth-1];
initial begin
            for(integer i = 0; i < depth; i = i+1 )
                mem [i] <= 0;
end
always @(posedge(clk)) begin
            if(wr_en)
                if(mem_data_size==0) mem [addr] <= wr_data[7:0];
                else if (mem_data_size==1) {mem [addr+1], mem [addr]} <= wr_data[15:0];
                else if (mem_data_size ==2) {mem [addr+3], mem [addr+2],mem [addr+1], mem [addr]} <= wr_data;
end
assign  rd_data = {mem [addr+3], mem [addr+2], mem [addr+1], mem [addr]};
endmodule
```

? also if( ) ?





4. Testbench

4. Testbench

- Direct Testcase (1):

addi $s0, $0, 0x0AF0 //2800	20100af0
addi $s1, $0, 0x04B0 //1200	201104b0
add $s2, $s0, $s1	02119020
addi $t0, $0, 0x0FA0 //Result	20080fa0
beq $s2, $t0, NEXT1	12480001
j ERROR	08000016
NEXT1: sub $s3, $s0, $s1	02119822
addi $t1, $0, 0x0640 //Result	20090640
beq $s3, $t1, NEXT2	12690000
j ERROR	08000016
NEXT2: and $s4, $s0, $s1	0211a024
addi $t2, $0, 0x00B0 //Result	200a00b0
beq $s4, $t2, NEXT3	128a0000
j ERROR	08000016

NEXT3: or $s5, $s0, $s1	0211a825
addi $t3, $0, 0x0EF0 //Result	200b0ef0
beq $s5, $t3, NEXT4	12ab0000
j ERROR	08000016
NEXT4: addi $t4, $0, 0x0064	200c0064
sw $s0, 50($t4)	ad900032
lw $t5, 50($t4)	8d8d0032
beq $s0, $t5, DONE	120d0002
ERROR: addi $s0, $0, 0XDEAD	2010dead
j SKIP	08000018
DONE: addi $s0, $0, 0XD08E	2010d08e
SKIP: nop	00000000

4. Testbench

- Randomized Testcase:
  Build enum for the main parts of your instruction to be randomized later.

```
////////////////////////////////////////////////////////////////////////////////
typedef enum logic [1:0] {R_TYPE, I_TYPE, J_TYPE} instr_type_e;

typedef enum logic [5:0] {r_type=0, j=2, beq=4, addi=8, lw=35, sw=43} opcode_e;

typedef enum logic [5:0] {add=32, sub=34, andd=36, orr=37, slt=42} funct_e;

typedef enum logic [4:0] {rs_t0=8,rs_t1,rs_t2,rs_t3,rs_t4,rs_t5,rs_t6,rs_t7=15,
rs_s0=16,rs_s1,rs_s2,rs_s3,rs_s4,rs_s5,rs_s6,rs_s7,rs_t8,rs_t9=25} rs_e;

typedef enum logic [4:0] {rt_t0=8,rt_t1,rt_t2,rt_t3,rt_t4,rt_t5,rt_t6,rt_t7=15,
rt_s0=16,rt_s1,rt_s2,rt_s3,rt_s4,rt_s5,rt_s6,rt_s7,rt_t8,rt_t9=25} rt_e;

typedef enum logic [4:0] {rd_t0=8,rd_t1,rd_t2,rd_t3,rd_t4,rd_t5,rd_t6,rd_t7=15,
rd_s0=16,rd_s1,rd_s2,rd_s3,rd_s4,rd_s5,rd_s6,rd_s7,rd_t8,rd_t9=25} rd_e;
////////////////////////////////////////////////////////////////////////////////
```

4. Testbench

- Randomized Testcase:

  Then, construct each instruction type.

```
typedef struct packed {
        opcode_e opcode;
        rs_e rs;
        rt_e rt;
        rd_e rd;
        logic [4:0] shamt;
        funct_e funct;
} R_TYPE_e;

typedef struct packed {
        opcode_e opcode;
        rs_e rs;
        rt_e rt;
        logic signed [15:0] immediate;
} I_TYPE_e;

typedef struct packed {
        opcode_e opcode;
        logic signed  [25:0] immediate;
} J_TYPE_e;
```

4. Testbench

- Randomized Testcase:

  Then, randomize each type, and do your constraints.

```
class transaction;
        rand logic [31:0] instruction;
        rand R_TYPE_e       R_TYPE_instruction;
        rand I_TYPE_e       I_TYPE_instruction;
        rand J_TYPE_e       J_TYPE_instruction;
        rand instr_type_e   option;

        constraint instr_constr {
            if(option==R_TYPE) { instruction==R_TYPE_instruction; }
            else if(option==I_TYPE) { instruction==I_TYPE_instruction; }
            else if(option==J_TYPE) { instruction==J_TYPE_instruction; }
        }

        constraint instr_dist_constr {
            option dist {R_TYPE:=60, I_TYPE:=40, J_TYPE:=10};
        }
endclass
```

4. Testbench

- Randomized Testcase:

  After that, build your testbench, creating the required clk, and write the randomized instruction inside a file to be put initially inside the instruction mem.

```
module mips_tbench();
    ////////////////////////////////////////
    parameter clk_period=10;
    logic clk;
    initial begin
            clk=0;
            forever
            #(clk_period/2) clk=~clk;
    end
    ////////////////////////////////////////
    transaction t;
    task write_instruction_file();
            fd=$fopen("mem.dat","w");
            for(int i=0;i<1024;i=i+1) begin
                assert(t.randomize() );
                $fdisplay(fd,"%h",t.instruction[7:0]);
                $fdisplay(fd,"%h",t.instruction[15:8]);
                $fdisplay(fd,"%h",t.instruction[23:16]);
                $fdisplay(fd,"%h",t.instruction[31:24]);
            end
            $fclose(fd);
     endtask
      initial $readmemh("mem.dat", mips_top_inst.instruction_mem_ins.mem);
endclass
```

5. Synthesis

5. Synthesis

- Edit XDC file:
- Create clock at clk port with 100 MHz.

```
## This file is a general .xdc for the ZYBO Rev B board
## To use it in a project:
## - uncomment the lines corresponding to used pins
## - rename the used signals according to the project


##Clock signal
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { clk }]
create_clock -add -name sys_clk_pin -period 20.00 -waveform {0 10} [get_ports { clk }]
```

Thank You